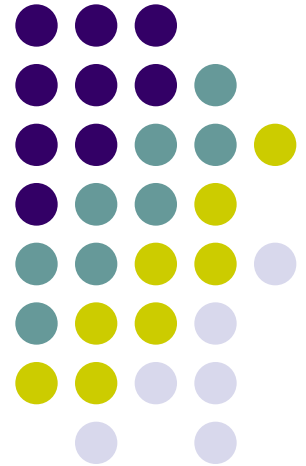


# php



**Teacher: Wafaa Alnbhany**

## **Lecture12**



# PHP Cookies

A cookie is often used to identify a user.

## **What is a Cookie?**

A cookie is often used to identify a user. A cookie is a small file that the server embeds on the user's computer. Each time the same computer requests a page with a browser, it will send the cookie too. With PHP, you can both create and retrieve cookie values.

## **How to Create a Cookie?**

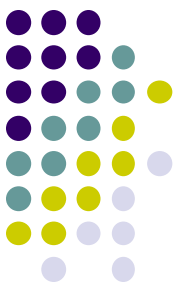
The `setcookie()` function is used to set a cookie.

**Note:** The `setcookie()` function must appear BEFORE the `<html>` tag.

### **Syntax**

```
setcookie(name, value, expire, path, domain);
```

## Example 1



In the example below, we will create a cookie named "user" and assign the value "Alex Porter" to it. We also specify that the cookie should expire after one hour:

```
<?php
setcookie("user", "Alex Porter", time()+3600);
?>
<html>
.....
```

**Note:** The value of the cookie is automatically URLencoded when sending the cookie, and automatically decoded when received (to prevent URLencoding, use `setrawcookie()` instead).

## Example 2

You can also set the expiration time of the cookie in another way. It may be easier than using seconds.

```
<?php
$expire=time()+60*60*24*30;
setcookie("user", "Alex Porter", $expire);
?>
<html>
.....
```

In the example above the expiration time is set to a month (*60 sec \* 60 min \* 24 hours \* 30 days*).

# How to Retrieve a Cookie Value?

The PHP `$_COOKIE` variable is used to retrieve a cookie value.

In the example below, we retrieve the value of the cookie named "user" and display it on a page:

```
<?php
// Print a cookie
echo $_COOKIE["user"];
// A way to view all cookies
```

```
print_r($_COOKIE);
?>
```

In the following example we use the `isset()` function to find out if a cookie has been set:

```
<html>
<body>
<?php
if (isset($_COOKIE["user"]))
    echo "Welcome " . $_COOKIE["user"] . "!<br />";
else
    echo "Welcome guest!<br />";
?>
</body>
</html>
```

# How to Delete a Cookie?

When deleting a cookie you should assure that the expiration date is in the past.

Delete example:

```
<?php
// set the expiration date to one hour ago
setcookie("user", "", time()-3600);
?>
```

## What if a Browser Does NOT Support Cookies?

If your application deals with browsers that do not support cookies, you will have to use other methods to pass information from one page to another in your application. One method is to pass the data through forms (forms and user input are described earlier in this tutorial).

The form below passes the user input to "welcome.php" when the user clicks on the "Submit" button:

```
<html>
<body>
<form action="welcome.php" method="post">
Name: <input type="text" name="name" />
Age: <input type="text" name="age" />
```

```
<input type="submit" />
</form>
</body>
</html>
```

Retrieve the values in the "welcome.php" file like this:

```
<html>
<body>
Welcome <?php echo $_POST["name"]; ?>.<br />
You are <?php echo $_POST["age"]; ?> years old.
</body>
</html>
```



# **PHP Sessions**

A PHP session variable is used to store information about, or change settings for a user session. Session variables hold information about one single user, and are available to all pages in one application.

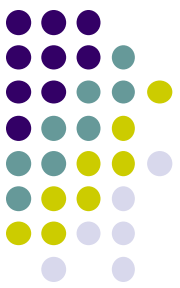
## **PHP Session Variables**

When you are working with an application, you open it, do some changes and then you close it. This is much like a Session. The computer knows who you are. It knows when you start the application and when you end. But on the internet there is one problem: the web server does not know who you are and what you do because the HTTP address doesn't maintain state.

A PHP session solves this problem by allowing you to store user information on the server for later use (i.e. username, shopping items, etc). However, session information is temporary and will be deleted after the user has left the website. If you need a permanent storage you may want to store the data in a database.

Sessions work by creating a unique id (UID) for each visitor and store variables based on this UID. The UID is either stored in a cookie or is propagated in the URL.

# Starting a PHP Session



Before you can store user information in your PHP session, you must first start up the session.

**Note:** The `session_start()` function must appear BEFORE the `<html>` tag:

```
<?php session_start(); ?>
<html>
<body>
</body>
</html>
```

The code above will register the user's session with the server, allow you to start saving user information, and assign a UID for that user's session.

# Destroying a Session

If you wish to delete some session data, you can use the `unset()` or the `session_destroy()` function.

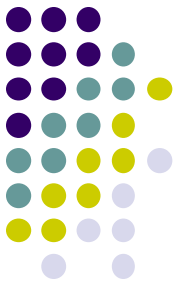
The `unset()` function is used to free the specified session variable:

```
<?php
unset($_SESSION['views']);
?>
```

You can also completely destroy the session by calling the `session_destroy()` function:

```
<?php
session_destroy();
?>
```

**Note:** `session_destroy()` will reset your session and you will lose all your stored session data.



# What is HTTP Authentication?

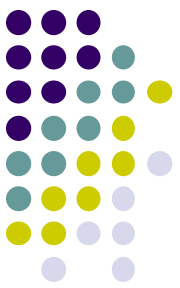
- HTTP Authentication is a way to restrict access to web pages.
- It requires users to enter a username and password to access protected content.
- In PHP, this is implemented using the `$_SERVER` superglobal and `header()` function.

# Types of HTTP Authentication

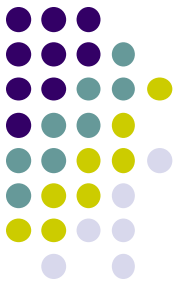


- .1Basic Authentication –Credentials are sent encoded in Base.64
- .2Digest Authentication –More secure, uses hashing for credentials.
- Basic Authentication is simpler and often used for testing or internal pages.

# Example noitacitnehtuA cisaB :1



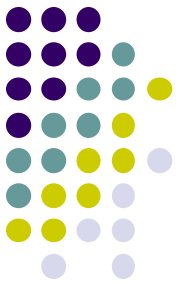
- `?>php`
- `if (!isset($_SERVER['PHP_AUTH_USER']))`
- `header('WWW-Authenticate: Basic realm="Restricted Area"');`
- `header('HTTP/1.0 401 Unauthorized');`
- `echo 'Authentication required;'`
- `exit;`
- `{else}`
- `if ($_SERVER['PHP_AUTH_USER'] == 'admin' &&`  
`$_SERVER['PHP_AUTH_PW'] == '1234`
- `echo '<h;<2h/>!nimdA ,emocleW<2`
- `{ else}`
- `header('HTTP/1.0 401 Unauthorized');`
- `echo 'Invalid username or password;'`
- `{`
- `{`
- `<?`
- Output: Displays a login popup; upon success, shows 'Welcome, Admin!'



# Example 2: Protecting a Page

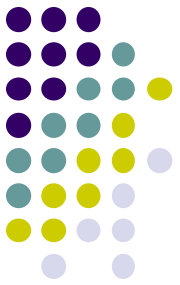
- `?>php`
- `if (!isset($_SERVER['PHP_AUTH_USER']) ||  
$_SERVER['PHP_AUTH_USER'] != 'teacher' ||  
$_SERVER['PHP_AUTH_PW'] != 'wafaa') {`
- `header('WWW-Authenticate: Basic realm="Private";')`
- `header('HTTP/1.0 401 Unauthorized');`
- `echo 'You are not authorized to access this page;'`
- `exit;`
- `{`
- `<?`
- `<h1>This is a protected page visible only to authorized users.</h1>`
- `<p>This is a protected page visible only to authorized users.</p>`

# How HTTP Authentication Works



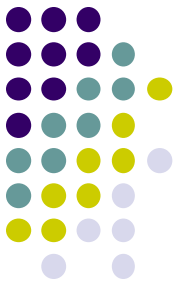
- 1. Browser requests the PHP page.
- 2. PHP checks for authentication data (`$_SERVER` variables).
- 3. If missing, PHP sends headers asking for credentials.
- 4. User enters username and password.
- 5. PHP verifies the credentials and grants or denies access.





# Security Notes

- Basic Authentication sends credentials in Base64, not encrypted.
- Always use HTTPS for secure communication.
- For sensitive systems, use sessions, tokens, or frameworks with built-in authentication.



# Summary

- HTTP Authentication restricts access using username and password.
- Implemented using `$_SERVER['PHP_AUTH_USER']` and `$_SERVER['PHP_AUTH_PW']`.
- 'WWW-Authenticate' header triggers browser login dialog.

Use HTTPS to secure transmitted credentials.